

Программирование на С

Лекция 9

Указатели и массивы

Указатель

Указатель — это переменная, которая хранит адрес памяти.

Что может хранить указатель

Указатель может хранить адрес:

- переменной;
- элемента массива;
- функции;
- объекта;
- другого указателя.

Инициализация указателей

```
int *countPtr = NULL;  
float *xPtr = NULL;  
float *yPtr = NULL;
```

или

```
float *xPtr = 0;  
float *yPtr = 0;
```

Указатель рекомендуется инициализировать сразу после объявления.

Неинициализированные указатели

```
float *xPtr;  
float *yPtr;  
  
int *countPtr;
```

Неинициализированный указатель содержит неизвестный адрес.

Использование такого указателя приводит к неопределенному поведению программы.

Операция разыменования

```
printf("%d\n", *yPtr);
```

Операция разыменования `*` возвращает значение объекта, на который указывает указатель.

Адресация и разыменование

```
int y = 5;  
  
int *yPtr = &y;  
  
printf("%d\n", *yPtr);
```

& получает адрес объекта.

* получает значение по адресу.

Указатель на указатель

```
int x = 5;  
  
int *px = &x;  
int **ppx = &px;  
int ***pppx = &ppx;
```

Указатель также является обычной переменной, поэтому можно получить адрес указателя.

Индексация и разыменование

```
assert(*x == x[0]);  
assert(*(x + i) == x[i]);
```

Индексация массива основана на арифметике указателей.

Операции с указателями

```
int a = 7;  
int *p = &a;  
  
printf("%d\n", a);  
printf("%d\n", *p);  
  
printf("%p\n", (void *)&a);  
printf("%p\n", (void *)p);  
  
printf("%p\n", (void *)&*p);  
printf("%p\n", (void *)&p);
```

7
7
0x...
0x...
0x...
0x...

- `p == &a`
- `*p == a`
- `&*p == p`
- `*&p == p`

Указатель на указатель

```
int a = 5;

int *p = &a;
int **pp = &p;

printf("%d\n", a);
printf("%d\n", *p);
printf("%d\n", **pp);

printf("%p\n", (void *)&a);
printf("%p\n", (void *)p);
printf("%p\n", (void *)*pp);

printf("%p\n", (void *)&p);
printf("%p\n", (void *)pp);
```

5
5
5

0x...
0x...
0x...

0x...
0x...

- `p` хранит адрес `a`
- `pp` хранит адрес `p`
- `**pp == a`

Массив и указатель

Имя массива можно использовать как адрес его первого элемента.

```
int arr[5];  
  
assert(arr == &arr[0]);
```

Имя массива — не указатель

```
int arr[5];
```

```
arr += 3;    // Ошибка
```

```
arr = NULL;  // Ошибка
```

```
int *p = arr;
```

```
p += 3;      // Хорошо
```

```
p -= 1;      // Хорошо
```

Указатель можно использовать как массив

```
#define SIZE 5

int arr[SIZE] = {1, 2, 3, 4, 5};

int *p = arr;

for (int i = 0; i < SIZE; i++) {
    printf("%d\n", p[i]);
}
```

Для доступа к элементам массива можно использовать как имя массива, так и указатель на его первый элемент.

Выход за границы массива

```
int arr[5];  
  
int *p = arr;  
  
p += 10;  
  
printf("%d\n", *p);
```

Выход за границы массива приводит к неопределенному поведению программы.

Арифметика указателей

Арифметические операции над указателями имеют смысл, только если указатель указывает на элемент массива.

Возможные операции:

- `p + n`
- `p - n`
- `p++`
- `p--`
- `p1 - p2`

Перемещение по массиву

```
int arr[5] = {10, 20, 30, 40, 50};  
  
int *p = arr;  
  
p++;  
p += 2;
```

После выполнения кода указатель будет указывать на `arr[3]` .

Размер элемента учитывается автоматически

```
double arr[5];  
  
double *p = arr;  
  
p += 3;
```

Компилятор автоматически учитывает размер типа.

Переход на следующий элемент массива означает переход на `sizeof(double)` байт.

Расстояние между указателями

```
int arr[5];  
  
int *p1 = &arr[1];  
int *p2 = &arr[4];  
  
printf("%td\n", p2 - p1);
```

3

Разность двух указателей равна количеству элементов между ними.

Индексация и арифметика указателей

```
arr[i]
```

ЭКВИВАЛЕНТНО

```
*(arr + i)
```

Операция индексации основана на арифметике указателей.

Двумерные массивы

Двумерный массив — это массив, элементами которого являются массивы.

```
int matrix[3][4];
```

`matrix` СОСТОИТ ИЗ:

- 3 строк;
- 4 элементов в каждой строке;
- 12 элементов всего.

Объявление двумерного массива

```
int matrix[3][4];
```

Первый размер — количество строк.

Второй размер — количество элементов в каждой строке.

	столбцы			
	0	1	2	3
строка 0				
строка 1				
строка 2				

Обращение к элементу

```
matrix[1][2] = 7;
```

```
matrix[номер строки][номер столбца]
```

Как и в обычном массиве, нумерация начинается с нуля.

Инициализация двумерного массива

```
int matrix[3][4] = {  
    {1, 2, 3, 4},  
    {5, 6, 7, 8},  
    {9, 10, 11, 12}  
};
```

1	2	3	4
5	6	7	8
9	10	11	12

Перебор двумерного массива

```
#define ROWS 3
#define COLS 4

int matrix[ROWS][COLS];

for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        printf("%d ", matrix[i][j]);
    }

    printf("\n");
}
```

Внешний цикл перебирает строки, внутренний — элементы строки.

Двумерный массив в памяти

```
int matrix[3][4];
```

Элементы располагаются в памяти последовательно:

```
matrix[0][0] matrix[0][1] matrix[0][2] matrix[0][3]  
matrix[1][0] matrix[1][1] matrix[1][2] matrix[1][3]  
matrix[2][0] matrix[2][1] matrix[2][2] matrix[2][3]
```

Сначала в памяти располагается первая строка, затем вторая и так далее.

Строка двумерного массива

```
int matrix[3][4];
```

`matrix[0]` — первая строка массива.

`matrix[1]` — вторая строка массива.

`matrix[2]` — третья строка массива.

Каждая строка сама является массивом из четырёх элементов типа `int`.

Указатель на первый элемент строки

```
int matrix[3][4];  
  
int *p = matrix[1];
```

Эквивалентно:

```
int *p = &matrix[1][0];
```

Теперь `p` указывает на первый элемент второй строки.

Индексация двумерного массива

```
matrix[i][j] == *(matrix[i] + j)
```

А поскольку

```
matrix[i] == *(matrix + i)
```

то:

```
matrix[i][j] == (*(matrix + i) + j)
```

Указатель на строку

```
int matrix[3][4];  
  
int (*p)[4] = matrix;
```

p — указатель на массив из четырёх элементов типа **int**.

```
p++;
```

После этого **p** указывает на следующую строку.

Как читать объявление

```
int (*p)[4];
```

1. `p` — имя переменной;
2. `*p` — `p` является указателем;
3. `(*p)[4]` — указатель ведёт на массив из четырёх элементов;
4. элементы массива имеют тип `int`.

Скобки необходимы из-за приоритета операций.

Массив указателей

```
int *rows[3];
```

`rows` — массив из трёх указателей на `int`.

```
int a[] = {1, 2, 3};  
int b[] = {4, 5};  
int c[] = {6, 7, 8, 9};
```

```
rows[0] = a;  
rows[1] = b;  
rows[2] = c;
```


Как читать объявление

```
int *rows[3];
```

1. `rows` — имя переменной;
2. `rows[3]` — массив из трёх элементов;
3. `*rows[3]` — каждый элемент является указателем;
4. указатели ведут на объекты типа `int`.

Двумерный массив и массив указателей

```
int matrix[3][4];
```

- все строки имеют одинаковую длину;
- элементы расположены в памяти подряд;
- тип строки — `int[4]`.

```
int *rows[3];
```

- указатели могут вести на разные массивы;
- строки могут иметь разную длину;
- сами массивы могут находиться в разных местах памяти.

Это разные типы

```
int matrix[3][4];  
  
int **p = matrix;    // Ошибка
```

Двумерный массив не является указателем на указатель.

`matrix` преобразуется в указатель на строку:

```
int (*p)[4] = matrix;
```

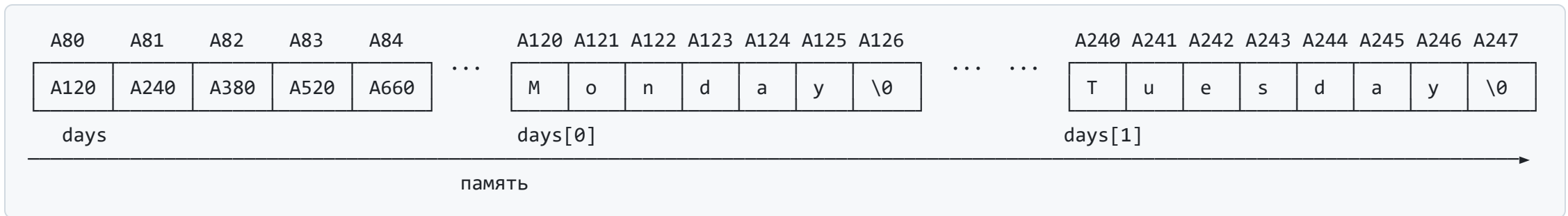
Массив указателей на строки

```
char *days[] = {  
    "Monday",  
    "Tuesday",  
    "Wednesday",  
    "Thursday",  
    "Friday"  
};
```

```
printf("%s\n", days[2]);
```

Wednesday

Каждый элемент массива `days` хранит указатель на первый символ строки.



```

char *c[] = {
    "ENTER",
    "NEP",
    "POINT",
    "FIRST"
};

char **cp[] = {
    c + 3,
    c + 2,
    c + 1,
    c
};

char ***cpp = cp;

printf("%s",    **++cpp);
printf("%s ",  *--*++cpp + 3);
printf("%s",    *cpp[-2] + 3);
printf("%s\n",  cpp[-1][-1] + 1);

```

Что будет напечатано?

Массивы указателей — ответ

POINTER STEP

`**++cpp`

POINT

`*--*++cpp + 3`

ER

`*cpp[-2] + 3`

ST

`cpp[-1][-1] + 1`

EP

Указатель на массив

Указатель на массив хранит адрес **целого массива фиксированного размера**.

```
int (*p)[3];
```

p — указатель на массив из трёх элементов типа **int**.

При увеличении **p** он перемещается к следующему массиву из трёх элементов.

```
p + 1
```

Смещение происходит на:

```
sizeof(int[3])
```


Указатель на строку матрицы

```
#define ROWS 2
#define COLUMNS 3

int matrix[ROWS][COLUMNS] = {
    {1, 2, 3}, {4, 5, 6}
};

int (*p)[COLUMNS] = matrix;
```

matrix

1	2	3	← p
4	5	6	← p + 1

Элементом массива `matrix` является не отдельное число, а целая строка типа `int[3]`.

Работа через указатель на массив

```
#define ROWS 2
#define COLUMNS 3

int matrix[ROWS][COLUMNS] = {
    {1, 2, 3}, {4, 5, 6}
};

int (*p)[COLUMNS] = matrix;

for (int i = 0; i < ROWS; ++i) {
    for (int j = 0; j < COLUMNS; ++j) {
        printf("%d ", p[i][j]);
    }

    printf("\n");
}
```

1	2	3
4	5	6

Через `p` можно
обращаться к элементам
матрицы так же, как
через имя двумерного
массива.

Одинаковый адрес — разные типы

```
printf("%p\n", (void *)matrix);  
printf("%p\n", (void *)matrix[0]);  
printf("%p\n", (void *)&matrix[0][0]);
```

Все три выражения обозначают один и тот же начальный адрес:

```
matrix  
matrix[0]  
&matrix[0][0]
```



1	2	3
4	5	6

Но после преобразования они имеют разные типы:

```
matrix      → int (*)[3]  
matrix[0]   → int *  
&matrix[0][0] int *
```

Разная арифметика указателей

```
int (*p)[3] = matrix;

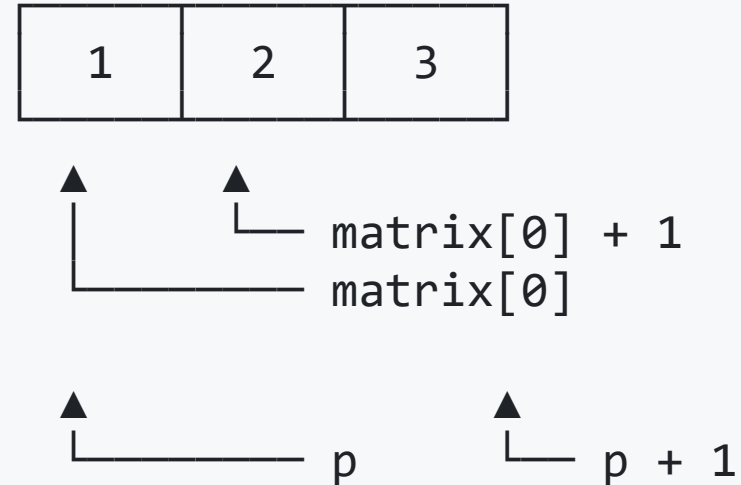
printf("%p\n", (void *)p);
printf("%p\n", (void *)(p + 1));

printf("%p\n", (void *)matrix[0]);
printf("%p\n", (void *)(matrix[0] + 1));
```

`p + 1`

переходит к следующей строке:

`sizeof(int[3])`



`matrix[0] + 1`

переходит к следующему числу:

`sizeof(int)`

Указатель можно изменить, имя массива — нельзя

```
int (*p)[3] = matrix;  
  
p++;           // допустимо  
  
matrix++;      // ошибка  
matrix[0]++;   // ошибка
```

p — переменная-указатель. Её значение можно изменить.

matrix и **matrix[0]** являются массивами, а не изменяемыми переменными-указателями.

Во многих выражениях имя массива преобразуется в указатель на первый элемент, но само имя массива указателем не становится.

Размер массива заранее неизвестен

```
int n;  
scanf("%d", &n);  
  
int arr[n];
```

Массивы переменной длины (Variable Length Arrays, VLA) появились в стандарте C99.

Начиная со стандарта C11, поддержка VLA стала необязательной для компиляторов. Многие проекты не используют VLA из-за ограниченной переносимости.

Как создать массив, если его размер становится известен только во время выполнения программы?

Если VLA использовать нельзя

```
int n;  
scanf("%d", &n);  
  
// Как создать массив?
```

```
int *arr = ???;
```

Программа должна запросить память во время выполнения.

Где взять память?

До этого момента память создавалась автоматически.

```
int a;  
int arr[100];
```

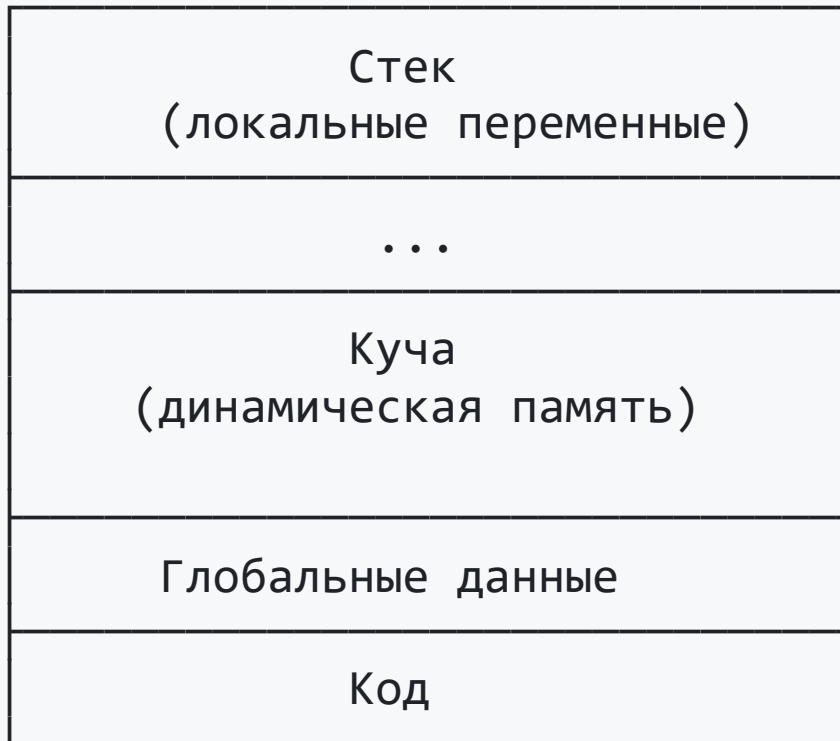
После выхода из функции эта память освобождается автоматически.

Иногда память должна существовать дольше или её размер заранее неизвестен.

Для этого используется динамическая память.

Память программы

Высокие адреса



Низкие адреса

Куча — область памяти, из которой программа может запрашивать память во время выполнения.

Выделение памяти

```
int *arr = malloc(n * sizeof(int));
```

`malloc()` выделяет область памяти и возвращает адрес её первого байта.

Если память выделить не удалось, функция возвращает:

NULL

Размер выделяемой памяти

```
int *arr = malloc(n * sizeof(int));
```

Количество памяти вычисляется как

число элементов × размер одного элемента

Например,

```
malloc(10 * sizeof(int));
```

выделяет память для десяти целых чисел.

Проверка результата

```
int *arr = malloc(n * sizeof(int));  
  
if (arr == NULL) {  
    printf("Ошибка выделения памяти\n");  
    return 1;  
}
```

Всегда проверяйте результат работы `malloc()`.

Работа с выделенной памятью

```
int *arr = malloc(5 * sizeof(int));
```

```
arr[0] = 10;  
arr[1] = 20;  
arr[2] = 30;
```

или

```
*(arr + 0) = 10;  
*(arr + 1) = 20;  
*(arr + 2) = 30;
```

После выделения памяти указатель используется так же, как имя массива.

Освобождение памяти

```
free(arr);
```

`free()` освобождает ранее выделенную динамическую память.

Каждому успешному вызову `malloc()` должен соответствовать вызов `free()`.

Что делает free()?

После malloc()



↑
arr

```
free(arr);
```

После free()

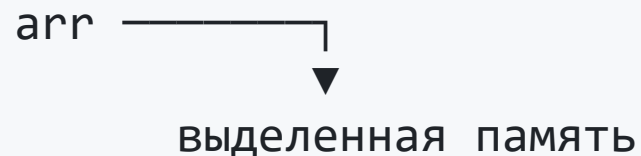


Указатель не изменяется

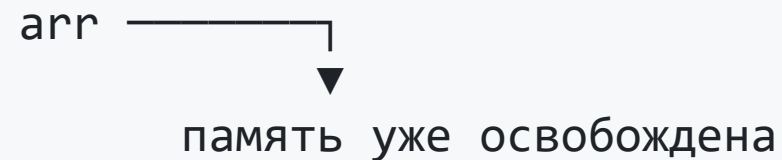
```
int *arr = malloc(5 * sizeof(int));  
  
free(arr);
```

После вызова `free()` значение указателя остаётся прежним.

До `free()`



После `free()`



После `free()` указатель становится висячим (dangling pointer).

Использование памяти после free()

```
int *arr = malloc(sizeof(int));  
  
free(arr);  
  
*arr = 10;
```

Использовать освобождённую память *нельзя*.

Результат работы программы не определён.

Повторное освобождение памяти

```
int *arr = malloc(sizeof(int));  
  
free(arr);  
free(arr);
```

Освободить одну и ту же область памяти дважды нельзя.

Такое поведение также не определено.

Как избежать ошибок

```
free(arr);  
arr = NULL;
```

После этого

```
if (arr != NULL) {  
    free(arr);  
}
```

становится безопаснее.

После `free()` полезно присвоить указателю значение `NULL`.

Утечка памяти

```
int *arr = malloc(100 * sizeof(int));  
  
return;
```

Память была выделена, но не освобождена.

Если программа теряет возможность обратиться к выделенной памяти, возникает утечка памяти (memory leak).

Потеря адреса

```
int *arr = malloc(100 * sizeof(int));
```

arr —————
 ↓
выделенная память

```
arr = NULL;
```

NULL ————— (адрес потерян)
 ↓
выделенная память

Если адрес потерян, освободить эту память уже невозможно.

Основные ошибки

Ошибка	Последствие
Забыли вызвать <code>free()</code>	Утечка памяти
Использование после <code>free()</code>	Неопределённое поведение
Повторный <code>free()</code>	Неопределённое поведение
Потеряли адрес	Утечка памяти

Все эти ошибки встречаются очень часто при работе с динамической памятью.

Последствия утечек памяти

- Программа начинает использовать всё больше памяти.
- Работа программы постепенно замедляется.
- Операционная система может завершить программу.
- Новые запросы памяти могут завершиться неудачей.

Даже небольшие утечки со временем могут стать серьёзной проблемой.

Небольшая утечка тоже опасна

```
while (1) {  
    malloc(1024);  
}
```

Каждый проход цикла выделяет ещё 1024 байта памяти.

Ни одна область памяти не освобождается.

Использование памяти будет непрерывно расти.

Когда утечки особенно опасны?

Для короткой программы небольшая утечка может быть незаметна.

Но многие программы работают:

- часами;
- сутками;
- месяцами;
- без перезапуска.

Чем дольше работает программа, тем опаснее становятся утечки памяти.

Функция calloc()

```
int *arr = calloc(n, sizeof(int));
```

calloc() выделяет память и заполняет её нулями.

Аргументы функции:

- количество элементов;
- размер одного элемента.

malloc() и calloc()

```
int *a = malloc(5 * sizeof(int));
```

malloc()



```
int *b = calloc(5, sizeof(int));
```

calloc()



Память после `malloc()` содержит неопределённые значения.

Изменение размера памяти

Иногда уже выделенной памяти становится недостаточно.

Например,

```
int *arr = malloc(5 * sizeof(int));
```

Позже программе требуется место уже для десяти элементов.

Для изменения размера используется функция `realloc()`.

Функция realloc()

```
arr = realloc(arr, 10 * sizeof(int));
```

realloc() изменяет размер ранее выделенной области памяти.

Новый размер задаётся в байтах.

Что делает realloc()

До realloc()



После realloc()



Содержимое уже существующих элементов сохраняется.

Адрес может измениться

До `realloc()`

`arr`



После `realloc()`

`arr`



Проверка результата realloc()

```
int *tmp = realloc(arr, 10 * sizeof(int));  
  
if (tmp != NULL) {  
    arr = tmp;  
}
```

Если `realloc()` не смог выделить память, исходная область памяти остаётся без изменений.

Динамический двумерный массив

Для двумерного массива необходимо выбрать способ хранения данных в памяти.

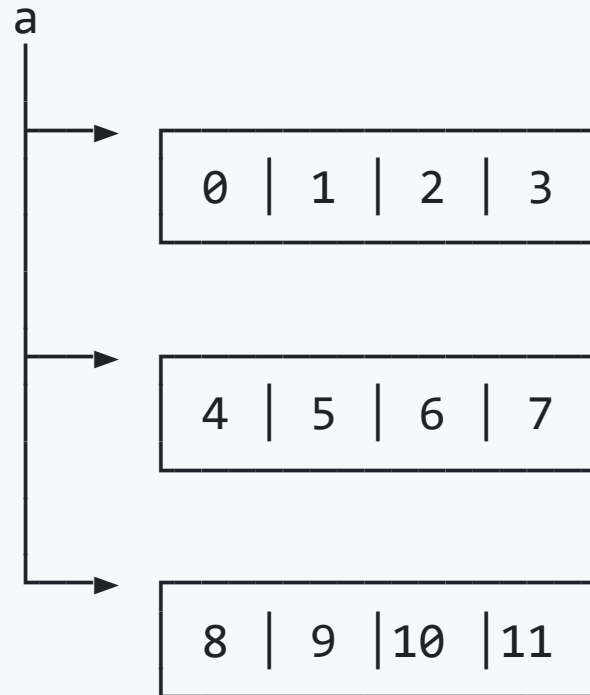
Сегодня рассмотрим три наиболее распространённых способа.

Способ 1. Массив указателей

```
int **a = malloc(rows * sizeof(int *));  
  
for (int i = 0; i < rows; i++)  
    a[i] = malloc(cols * sizeof(int));
```

Каждая строка выделяется отдельно.

Как выглядит память?



Каждая строка располагается независимо от остальных.

Освобождение памяти

```
for (int i = 0; i < rows; i++)  
    free(a[i]);  
  
free(a);
```

Необходимо освободить каждую строку, а затем массив указателей.

Достоинства и недостатки

Плюсы

- строки могут иметь разную длину;
- удобно менять отдельные строки.

Минусы

- много вызовов `malloc()` ;
- память не является непрерывной;
- освобождение памяти сложнее.

Способ 2. Один большой массив

```
int *a = malloc(rows * cols * sizeof(int));
```

Вся матрица хранится как один непрерывный блок памяти.

Как выглядит память?

a
|
▼

0	1	2	3	4	5	6	7	8	9	10	11
---	---	---	---	---	---	---	---	---	---	----	----

Все элементы располагаются подряд.

Доступ к элементам

Элемент с индексами

(i, j)

имеет номер

$i * \text{cols} + j$

$a[i * \text{cols} + j]$

Освобождение памяти

```
free(a);
```

Достаточно одного вызова `free()` .

Достоинства и недостатки

Плюсы

- один вызов `malloc()` ;
- один вызов `free()` ;
- непрерывная память;
- хорошая производительность.

Минусы

- необходимо вычислять индекс элемента.

Способ 3. Указатель на массив

```
int (*a)[cols] =  
    malloc(rows * sizeof(*a));
```

Тип указателя соответствует одной строке массива.

Использование

```
a[i][j]
```

Работает точно так же, как для обычного двумерного массива.

Никаких дополнительных вычислений писать не требуется.

Как выглядит память?

a
|
▼

row0		row1		row2		row3		...	
------	--	------	--	------	--	------	--	-----	--

Каждая строка содержит cols элементов.

В памяти хранится один непрерывный блок.

Освобождение памяти

```
free(a);
```

Как и во втором способе, достаточно одного вызова `free()` .

Сравнение способов

Способ	Непрерывная память	Один <code>free()</code>	Разная длина строк
<code>int **</code>	✗	✗	✓
<code>int *</code>	✓	✓	✗
<code>int (*)[N]</code>	✓	✓	✗

Какой способ выбрать?

Все три способа используются на практике.

- `int **` — если строки могут иметь разную длину.
- `int *` — если нужен полный контроль над памятью.
- `int (*)[N]` — если требуется обычная прямоугольная матрица.

Для прямоугольных матриц чаще всего используют второй или третий способ.